Same-State Counterfactual Forks: How Quantization-Induced Action Changes Become Task Consequences in Closed-Loop Agents

Anonymous ACL submission

Abstract

Quantization changes some of an LLM agent's actions. In a closed loop, one changed action alters the user's replies, the tool results and every later decision, so end-task success cannot show how such changes become task consequences. We study this with *same-state counterfactual forks*: at the first divergence between a BF16 agent and its quantized counterpart, we replay the identical state and cross {current action: BF16 or quantized} × {continuation controller: BF16 or quantized}. Serving is deterministic, so every no-op replay is bit-exact. The four branches give signed estimands: rescues r, damages d, outcome turnover $M = r + d$, and the gain $G = r - d$ from replacing the action at the forked state, which is not the end-to-end BF16-versus-quantized success difference. On Qwen3-4B and Qwen3-8B in τ^2-bench (85 tasks × 20 nonce blocks), replacing a W4 agent's divergent action changes the task outcome in 15.5–20.9% of task-block units, in both directions, while G ranges from -1.9 to $+2.9$ pp. Turnover concentrates in fragile tasks whose direction repeats across nonces; this is a retrospective association on development data, and its pre-registered holdout test is pending. On outcome-independent shared states, 39 of the 56 outcome-changing differences are same-structure text differences rather than tool or argument differences. Injected local defects cause two-sided turnover that we cannot separate from a generic process perturbation, so local defects and final failures do not map one-to-one. [TODO HOLDOUT: one sentence with the confirmatory G_{all} reading once the frozen telecom holdout is analysed]

1 Introduction

Quantized models are a standard way to deploy LLM agents, and quantized agents are usually evaluated as quantized classifiers are: compare the end-task score with that of the full-precision reference. In a closed-loop benchmark such as τ^2-bench (Barres et al., 2025), however, the agent talks to a simulated user and calls tools until the episode ends. Once one action differs, the user's reply, the tool results and every later decision can differ too. This paper asks **how quantization-induced changes in full actions become task consequences through the interaction and the subsequent execution.**

Why end-to-end comparisons cannot answer this. An end-to-end comparison runs the two agents independently and compares success. Such comparisons can hide large changes behind a flat average (Dutta et al., 2024; Jang et al., 2026). They also cannot attribute a change. A success difference between two independent trajectories mixes the effect of the first changed action with everything both agents do afterwards. Our own end-to-end comparisons are also inconclusive. On the checkpoints, tasks and nonce blocks we fork, quantized-minus-BF16 success ranges from -0.2 to $+4.2$ pp across four W4 cells. Yet task-resampled 95% intervals are about ±6–7 pp wide, and an equivalence test at ±3 pp fails in every cell (Appendix B). We therefore do not claim that quantized and BF16 agents succeed equally.

What we do. We intervene at a single state. Along the quantized agent's own trajectory, we find the first decision at which BF16 (B) and the quantized agent (Q) produce different full actions from the identical saved history. We then replay that state four times, executing either agent's action and continuing with either agent to the end (Figure 1). Crossing {current action} × {continuation controller} separates three questions. Does the current action matter? Does the controller that continues matter? Do the two interact? The pair $(Y_{\text{BQ}}, Y_{\text{QQ}})$ gives rescues (r), damages (d), turnover $M = r + d$ and the gain $G = r - d$. G is the gain from replacing the action at the forked

state and continuing with Q. It is not the end-to-end BF16-versus-quantized success difference.

What we find. The evidence comes in three layers, with decreasing strength.

1. **A two-sided consequence atlas** (§4). On the τ^2-bench retail/airline panel, replacing a W4 agent's first divergent action flips the task outcome in 15.5–20.9% of task-block units, in both directions. The fixed-pool G lies between -1.9 and $+2.9$ pp. On average over divergent states, the action, continuation and interaction terms are mostly indistinguishable from zero. Turnover concentrates in fragile tasks whose direction repeats across independent nonce blocks. The concentration is a retrospective conditional association on development data, in 6 configurations $\times$ 2 directions; its pre-registered holdout test has not been run ([TODO HOLDOUT: confirmatory reading]).
2. **Shared-state consequence structure** (§5). On ordinary states sampled independently of outcome, 39 of the 56 outcome-changing differences (≈69.6%) are same-structure text differences, such as talk versus talk. Only 5 of the 56 involve a different tool or different arguments. Different quantizers co-flip the same shared states with different actions, a single-block lead from 5 distinct states.
3. **Bounding results** (§6). Injected local defects produce heterogeneous two-sided turnover of the same size as a process-perturbation control, with no error-specific effect detected. An LLM-annotated process audit shows that local defects and final failure do not correspond one-to-one.

Contributions. (1) A same-state four-branch counterfactual protocol for closed-loop agents, with signed estimands, bit-exact replay acceptance and block-level inference. (2) A multi-axis atlas covering weight, KV-cache and stress quantization, three benchmarks and two model sizes. It shows large two-sided turnover at the forked state and its concentration in fragile tasks; that concentration is pre-registered for confirmation on a protected holdout. (3) Shared-state evidence that most consequential differences are full-language differences. We also report a controlled intervention and a process audit that bound the "local defect → failure" story. We report withdrawn claims, including a W×KV interaction that did not replicate, in Appendix E.

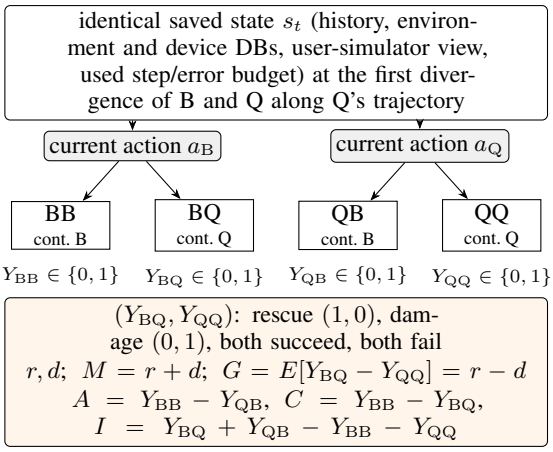


Figure 1: The same-state four-branch fork (schematic, not data). B = BF16 agent; Q = quantized agent. Branch XY executes the current action a_X as the agent's next step, and controller Y then continues to the end with the original remaining budget. QQ re-executes Q's original episode, and its bit-exact reproduction is the no-op check. $G = A + I$ is the action effect under Q's continuation; it is not the end-to-end success difference. Estimands are defined in §3.2.

2 Related Work

Quantized agents and flat scores. *Flat Score, Amplified Failures* (Jang et al., 2026) is the closest study in setting. It compares 16-, 8- and 4-bit agents on τ^2-bench retail and telecom. Scores stay nearly flat while process errors rise, a tolerant error budget masks the damage, and the failure set is nearly the same at every precision (rank correlation ≥ 0.94). That study compares whole episodes run independently, and its counterfactual re-scores episodes under a smaller error budget. We instead branch from identical states, which separates the changed action from what follows. We also measure signed outcome flips, rescues as well as damages, which an error-volume analysis does not show. Our "fragile tasks shared across quantizers" (§4) overlaps with its stable failure set. What we add there is that the *direction* of a shared flip is close to a coin toss across quantizers. Two-sided flips under compression were first emphasised on static tasks (Dutta et al., 2024). Aggregate agent-compression benchmarks report scores (Dong et al., 2025). Precision sensitivity is localised to steps (Li et al., 2026c), phases (Lu et al., 2026) or task types (Zhang et al., 2026). DMR (Li et al., 2026c) learns per-step BF16/W4 routing from KL supervision and episode reward. Our G is the outcome counterfactual that such a router would harvest at one decision; we measure

it rather than learn a proxy for it. Lotfi et al. (2026) give a token-level mechanism for quantized reasoning models. High-KL positions coincide with high entropy, and there the quantized model oversamples markers such as "wait". A logit penalty on those markers both repairs the damage and validates the mechanism. Our analogous token-level diagnosis at shared states is pending (§5).

Counterfactual replay and forking. Single-step action swaps that let the same policy continue are now common. They are used for training (Li et al., 2026a,b), for attribution (Shah, 2026) and for locating the decisive error (Zhang et al., 2025). Each fills one cell of our design: an alternative action under a fixed continuation. *The Replay Gap* (Gonuguntla, 2026) forks live SWE-agent trajectories and swaps the *continuation* model, comparing Qwen3-4B (FP8) with Qwen3-14B (AWQ), against same-model control forks. Its 359 control forks had 0 success-label flips. Its FP8-served controls diverged in action sequence on 90–96% of forks. We do not claim a lower outcome noise floor than that work. The difference is the design. We fork at a precision-induced divergence between two precisions of the same weights. We cross the action with the continuation controller. We also require every no-op replay to be bit-exact in prompt tokens, messages and database hashes (§3.3). None of the 32 closest works we checked crosses both axes or defines r, d, M, G as estimands. Novelty therefore rests on the factorial design and the shared-state results, not on forking itself.

Evaluation noise and user simulators. Serving numerics change greedy trajectories. In multi-turn tool use, prefix caching changed the trajectory in 36.2% of episodes at 16-bit and 75.0% at 4-bit; with the cache off, runs were bit-identical (Patodiya, 2026). Agentic evaluations also vary from run to run (Bjarnason et al., 2026; Chen et al., 2026). The user simulator matters too. Changing the user model moves τ-bench retail success by up to 9 points (Seshadri et al., 2026) and reward by 15.2 points (Jain and Sandhu, 2026). Because most of our consequential differences are text, the simulator is a plausible mediator. A second-simulator control is pending (§5).

3 Same-State Four-Branch Forks

3.1 Setting

Tasks and agents. We use τ^2-bench in text mode. The development panel has 58 retail and 27 airline tasks plus 80 telecom development tasks. A further 34 telecom tasks form a hold-out that no agent ran before its protocol was frozen. The agents are Qwen3-4B-Instruct-2507 (q4) and Qwen3-8B with thinking disabled (q8). The user simulator is a single greedy Qwen3-30B-A3B model shared by all agents. Retail and airline success is the final-database match. Telecom uses the official task-basis success. Without a user simulator, we also run BFCL multi-turn (four categories) and ALFWorld (out-of-distribution split) with one run per item (Appendix A).

Quantization. Q is a simulated (fake-quantized) checkpoint served with BF16 compute on the same path as B. Weight cells are GPTQ (Frantar et al., 2023) and AWQ (Lin et al., 2024) at W4. The KV-cache cell is KV8. Stress cells are GPTQ-W3, RTN-W3 and a Q/K-rotated KV4 cache. Because quantization is simulated, we make no speed or memory claim.

Deterministic serving and blocks. B and Q run on vLLM with batch-invariant kernels and deterministic compilation, in separate server processes, so no KV or prefix cache is shared across weights. A numerics canary is checked at every server start. A *block* is one closed-loop run of Q over the task pool under one prompt nonce, a random session-identifier line in the system prompt. Independent nonces give independent blocks, and blocks are the replicate unit.

3.2 Fork states and branches

For each task, block and quantized cell, we walk along Q's closed-loop trajectory. Before Q's decision t we restore the saved state s_t. It comprises the full message history; the environment and user-device databases, rebuilt by strict replay of every recorded tool call; the user simulator's visible conversation; and the used step and error budget. B generates its full action from the same request, and the prompt token ids B's server receives must equal those Q received. The fork state is the first t at which the two actions differ in class (tool call, text, stop), tool name, call count or order, or normalised arguments. Differences in text only are recorded but are not fork states in this

layer; §5 covers them. A task without such a divergence stays in the denominator with zero effect. The state is frozen to disk before any branch runs.

From s_t, branch XY executes a_{X} as the agent's next step; the real environment executes it or the real simulator answers it, and it consumes one step. Controller Y then continues to the end with the original remaining budget. Outcomes $Y_{\mathrm{XY}} \in \{0, 1\}$. QQ reproduces Q's original episode.

Estimands. On the full task × block pool, with non-divergent units contributing zero,

$$r = P(Y_{\mathrm{BQ}}{=}1, Y_{\mathrm{QQ}}{=}0),$$
$$d = P(Y_{\mathrm{BQ}}{=}0, Y_{\mathrm{QQ}}{=}1),$$

$M = r + d$ (outcome turnover) and $G_{\mathrm{all}} = r - d = E[Y_{\mathrm{BQ}} - Y_{\mathrm{QQ}}]$. G_{all} is the gain from replacing Q's action at the fork state while Q continues. It is *not* the end-to-end BF16-versus-Q success difference, which compares two independent trajectories (Appendix B). On divergent states, the 2×2 decomposition is

$$A = Y_{\mathrm{BB}} - Y_{\mathrm{QB}} \quad \text{(action, B continues)},$$
$$C = Y_{\mathrm{BB}} - Y_{\mathrm{BQ}} \quad \text{(controller, B acts)},$$
$$I = Y_{\mathrm{BQ}} + Y_{\mathrm{QB}} - Y_{\mathrm{BB}} - Y_{\mathrm{QQ}},$$

the interaction, so $G = A + I$, $H = Y_{\mathrm{QB}} - Y_{\mathrm{QQ}} = C + I$ and $Y_{\mathrm{BB}} - Y_{\mathrm{QQ}} = A + C + I$. A *strict reversal* ($A \cdot G < 0$) is a state where the better action under one controller is the worse action under the other. Appendix A gives the full four-way composition of $(Y_{\mathrm{BQ}}, Y_{\mathrm{QQ}})$ per cell. A, C and I describe where an outcome change at the fork comes from. They do not judge whether an action is wrong; B is a reference that can rescue or damage.

3.3 Replay acceptance

Causal reading of the branches requires replay to be exact. Before any development outcome was analysed, an engineering gate on real divergence states in retail, airline and telecom passed every check. Q→Q no-ops matched in 20/20 states and B→B no-ops in 10/10, identical in every prompt-token hash, message, live and replayed database hash, termination, reward and budget counter. A same-checkpoint fork (B and Q handles on two servers of one checkpoint) gave four identical branches in 6/6 states. Branches run in reverse order and concurrently gave identical signatures in 10/10 states, plus 3/3 in dual-control telecom. Live budget counters equalled the recount in all 80 gate branches. In production, a sampled QQ no-op passed 924/924 units of the development analysis, and the outcome-independent bank of §5 passed 135/135 no-ops. The replay noise floor of the fork is therefore exactly zero. This is a property of our serving, which closes the prefix-cache channel (Patodiya, 2026); we do not claim it as an advantage over prior forks.

3.4 Inference

All tasks of a block share its nonce, and B and Q share each state, so we compute each statistic per complete shared block. Primary intervals resample whole blocks: a block bootstrap with at least 10 blocks, otherwise a t interval over block means. These treat the task pool as fixed. A crossed task × block bootstrap is the secondary interval for new tasks. More nonce blocks narrow only the first, so we never let it stand in for the second. A block with any missing unit leaves the analysis.

4 Layer 1: A Two-Sided Consequence Atlas

Two-sided turnover at the fork. Table 1 reports every cell with its denominator. On the τ^2-bench panel (85 tasks × 20 complete blocks), replacing a W4 agent's first divergent action changes the task outcome in $M = 15.5$–20.9% of units. Rescues and damages are of similar size: $r = 6.8$–11.4 and $d = 7.5$–10.1 pp. The fixed-pool G_{all} lies between -1.9 and $+2.9$ pp. Its intervals exclude zero in two cells, negative for q4 GPTQ-W4 and positive for q8 GPTQ-W4, and none of the crossed intervals do. Pooled over the four W4 cells, G_{all} is $+0.7$ $[-0.3, +1.6]$ pp with $M = 18.6\%$. KV8 turns over fewer outcomes ($M = 6.5$–8.1%). Stress cells reach similar M with only three blocks. The external benchmarks show the same two-sided pattern, with W4 turnover of 5.2–24.6% of items; G_{all} covers zero except for BFCL q4 AWQ-W4. Small net values therefore coexist with substantial turnover in both directions. G_{all} is a quantity at the forked state. How end-to-end success compares is a separate question, and our end-to-end intervals are too wide to call it small (Appendix B).

What the 2×2 adds. On divergent states, the mean action, controller and interaction terms are mostly indistinguishable from zero (Table 1). The clearest exception is q8 GPTQ-W4. There B's action helps under B's continuation ($A = +5.0$ $[+3.2, +6.8]$), and B's continuation helps after

	Cell	tasks×blocks	r	d	M	G_{all} [fixed pool]	G_{all} [crossed]	A	C	I
τ^2-bench W4	q4 GPTQ-W4	85×20	6.8	8.7	15.5	−1.9 [−3.7, −0.1]	[−6.8, +2.7]	−3.2	+0.5	+0.3
	q4 AWQ-W4	85×20	9.8	10.1	19.9	−0.3 [−2.2, +1.6]	[−5.4, +5.2]	−1.7	−0.4	+1.3
	q8 GPTQ-W4	85×20	10.4	7.5	17.9	+2.9 [+1.4, +4.4]	[−2.9, +9.2]	+5.0†	+2.1†	−1.7
	q8 AWQ-W4	85×20	11.4	9.5	20.9	+1.9 [−0.6, +4.3]	[−3.9, +8.1]	+1.9	−1.5	+0.3
τ^2-bench KV8	q4 KV8	85×22	3.1	3.4	6.5	−0.3 [−1.7, +1.0]	[−3.0, +2.3]	+1.3	−0.2	−2.4
	q8 KV8	85×22	4.3	3.7	8.1	+0.6 [−0.4, +1.6]	[−1.9, +3.0]	+2.8†	+1.2	−1.5
τ^2-bench stress (t, k=3)	q4 GPTQ-W3	85×3	9.0	10.2	19.2	−1.2 [−7.0, +4.7]	[−8.6, +5.9]			
	q4 KV4-rot	85×3	11.8	7.1	18.8	+4.7 [−4.1, +13.5]	[−3.1, +12.9]			
	q8 GPTQ-W3	85×3	10.2	8.2	18.4	+2.0 [−12.5, +16.4]	[−6.3, +11.8]			
	q8 KV4-rot	85×3	11.8	8.6	20.4	+3.1 [−14.5, +20.8]	[−5.9, +12.9]			
	q8 RTN-W3	85×3	3.1	3.5	6.7	−0.4 [−3.8, +3.0]	[−4.7, +3.9]			
telecom dev	q8 GPTQ-W4	80×6			14.8	+2.7				
	q8 AWQ-W4	80×6			12.3	−0.6				
external W4 (items)	ALFWorld q8 GPTQ-W4	134	9.0	12.7	21.6	−3.7 [−11.2, +4.5]	item-level			
	ALFWorld q8 AWQ-W4	134	10.4	14.2	24.6	−3.7 [−11.9, +4.5]	item-level			
	BFCL q4 AWQ-W4	591	5.6	9.1	14.7	−3.6 [−6.6, −0.5]	item-level			
	BFCL q8 GPTQ-W4	788	2.9	2.3	5.2	+0.6 [−0.9, +2.3]	item-level			
	BFCL q8 AWQ-W4	788	4.1	2.7	6.7	+1.4 [−0.3, +3.3]	item-level			

Table 1: Consequence atlas (development data; pp). r, d, M and G_{all} are on the full task×block pool, with non-divergent units counted as zero. G_{all} is the gain from replacing Q's first divergent action while Q continues; it is *not* the end-to-end success difference (Appendix B). Fixed-pool intervals: block bootstrap (≥10 blocks) or t over block means. Crossed: task×block bootstrap for new tasks. A, C and I are means over divergent states (sign convention of §3.2); †: the 95% interval excludes 0 (all intervals in Table 6); not shown for cells with 3 blocks. Telecom development rows are point estimates from a separate fit (q8 only, 6 blocks). External rows have one run per item and item-level intervals. BFCL q4 GPTQ-W4 is missing from this committed snapshot ([TODO EXT-REFRESH: commit the newer external analysis output and refresh these rows]). Sources: `analysis/fork/out/dev_all_20261001T070326Z.md`, `holdout_k_planning_20261001T144122Z.md`, `ext_20260930T221613Z.md`.

B's action ($C = +2.1$ [+0.2, +4.0]). Interaction is frequent at the state level: $I \neq 0$ on about 15–30% of divergent states. Its mean interval still covers zero in every W4 cell. Strict reversals, where the better action under one controller is the worse under the other, occur on 1.2–4.2% of divergent states. At the level of means, then, neither the current action nor the continuation controller dominates. Consequences are state-specific and two-sided, which motivates looking at *which* states and *which* differences carry them.

Concentration in fragile tasks (exploratory, development). Outcome flips are not spread evenly. The per-task dispersion of M is 4.0–8.7 times its binomial null in the W4 and KV8 cells. The top 20% of tasks hold 57–71% of all flips. We split the 20 blocks into two disjoint halves of 10 nonces. Task-level M correlates across halves (Spearman 0.52–0.86). The direction repeats too. Tasks that were net-rescue in one half have a mean G in the other half that is 14–51 pp higher than that of tasks that were net-damage, in all 12 cell-directions. A Freedman–Lane test conditions on measured baselines: QQ success propensity and BF16 success, plus trajectory length in a post-hoc variant. With them, one half's task rate still predicts the other's in all 6 configurations × 2 directions for both M and G (Holm-adjusted). This is a *retrospective conditional association* on viewed development data. It is carried by retail. Airline alone gives 8 of 12 for M (5 after Holm), and 3 of 12 under a heteroscedasticity-robust wild bootstrap. Direction persistence beyond flip opportunity holds more often for rescues than for damages, in 11 versus 6 of the 12 cell-directions. Different quantizers flip the same tasks: the task-identity null is rejected in 7 of 8 quantizer pairs. When both flip, however, they agree in direction only 42–81% of the time. This overlaps with the stable failure set of Jang et al. (2026). What is new here is the signed, two-sided view: shared fragility, but with near-random relative direction across quantizers.

Pre-registered confirmation (pending). Before any holdout unit was run, we froze a confirmatory protocol (sha256 `902892dd. . .`). It covers 34 unseen telecom tasks × 12 fresh nonce blocks × 3 cells (q4 GPTQ-W4, q8 GPTQ-W4, q8 AWQ-W4, weighted 1/2, 1/4, 1/4). The primary endpoint is G_{all}, read on a

crossed task × block 90% interval against ±3 pp bounds. The secondaries are two-sidedness ($r > 0$ and $d > 0$) and cross-half repeatability of task fragility. The repeatability test compares real task pairings with persona-preserving random pairings. It makes no claim beyond task difficulty. On development data only, the planned crossed interval width at $K = 12$ is about 4.1 pp. [TODO HOLDOUT: G_{all} with crossed 90% CI and its reading; S1 r/d lower bounds; S2 ρ and permutation p; structural exclusions; from `analysis/fork/analyze_holdout_confirm.py` under prereg sha 902892dd]. Development results are not retroactively confirmed by the holdout. [TODO Q14: paired BF16/W4 forks on Qwen3-14B τ^2-bench retail/airline, if they finish; otherwise state the 4B/8B scope only]

5 Layer 2: Shared-State Consequence Structure

The fork states of §4 are selected: each is the first divergence, and text-only differences before it are not intervened on. To learn *which* differences carry consequences at ordinary states, we built Bank U, a bank of states sampled independently of any outcome.

Design. Each Bank U state is one uniformly drawn decision per task, with the draw fixed by a hash, and the manifest was frozen before any generation. U1 contains states from each quantized cell's own trajectories. U2B contains BF16-sourced states, and U2Q a small set of Q-sourced shared states. Every configuration of a model receives the identical full U2 state, which allows same-state comparison across quantizers. Every state runs all four branches unless a_{B} and a_{Q} are token-identical, and the suffix effect is never imputed as zero. A frozen 25% subsample of U2 is also continued by one common controller Q0, the GPTQ-W4 checkpoint of the same model. This puts every quantizer's action under the same continuation. Here we report block f01: 85 U1, 85 U2B and 20 U2Q states per cell, for q4/q8 × {GPTQ-W4, AWQ-W4, KV8}. With one block, the intervals resample tasks only.

Consequence rates at ordinary states. At ordinary states, quantization changes the outcome less often than at first divergences. Per-state M is 8–12% for q4 W4 and 0–5% for q8 and KV8, and G covers zero everywhere (Table 2). Exact action agreement is 0.44–0.85 per cell and sub-bank. Identical actions never change the outcome (class (i), Table 3), as the deterministic protocol implies.

Cell	U1 (n=85)	U2B (n=85)	U2Q (n=20)
q4 GPTQ-W4	.118 (5/5)	.094 (5/3)	.100 (1/1)*
q4 AWQ-W4	.106 (3/6)	.082 (3/4)	.100 (1/1)
q4 KV8	.024 (0/2)	.047 (1/3)	0 (0/0)
q8 GPTQ-W4	.012 (0/1)	.024 (0/2)	0 (0/0)*
q8 AWQ-W4	.047 (2/2)	.024 (1/1)	0 (0/0)
q8 KV8	.047 (1/3)	.012 (1/0)	0 (0/0)

Table 2: Bank U, block f01: per-state turnover M (rescues/damages), task-equal, one state per task. G lies within ±0.035 in every cell, and every interval includes 0. *GPTQ-W4's U2Q states are its own U1 states, with concordant outcomes on 20/20 duplicates; they are counted once in cross-sub-bank totals. One block, so the intervals reflect task resampling only. Source: `analysis/fork/out/bank_u_f01_20261001T134456Z.md`.

Model:sub	(i) identical	(ii) same structure, different text	(iii) different tool or args	(iv) act/talk differs
q4:U1	141: 0	93: 14 (6/8)	8: 0	13: 7 (2/5)
q8:U1	173: 0	59: 7 (3/4)	13: 2 (0/2)	10: 0
q4:U2B	141: 0	91: 16 (8/8)	11: 2 (1/1)	12: 1 (0/1)
q8:U2B	172: 0	51: 1 (0/1)	11: 1 (1/0)	21: 3 (1/2)
q4:U2Q	34: 0	21: 3 (1/2)*	2: 0	3: 1 (1/0)
q8:U2Q	50: 0	5: 0	2: 0	3: 0
flips (rows)	0	41	5	12
flips (dedup.)	0	39	5	12

Table 3: What kind of difference changes the outcome. Entries are "state rows: outcome-changing rows (rescues/damages)", pooled over the three cells of a model within each sub-bank. The 58 outcome-changing rows deduplicate to 56: the two q4 GPTQ-W4 U2Q class-(ii) flips (*) repeat its U1 states. Class (ii) is almost entirely talk versus talk. Source: `analysis/fork/out/bank_u_f01_20261001T134456Z.md`; deduplication: `docs/REPORT_GPT61_20261001_0915.md` (correction 1).

Which differences matter: mostly text. After deduplication, 39 of the 56 outcome-changing differences (≈69.6%) are class (ii): the two actions have the same tool structure but different text, nearly always two different natural-language replies (Table 3). Only 5 of the 56 involve a different tool name, call count or argument. This is a *composition share*. Text differences dominate the flips mainly because they are common. Per instance, a text difference flips the outcome in 0–18% of rows depending on model and sub-bank. An act/talk difference flips in 0–54%, though on small denominators. Simple content features do not separate flipping from non-flipping text pairs: a question, a confirmation request, a claim of com-

pleted action or a new entity on one side only. Only a larger length difference co-occurs with flips, and only for q4 (median $|\Delta\text{len}|$ 118 vs. 71 characters in U1; 97 vs. 48 in U2B). This is descriptive; no test was run. The first-divergence atlas of §4 does not intervene on text-only differences. On ordinary states, such differences account for most outcome changes.

Shared fragility across quantizers (a lead). On the 85 shared q4 U2B states, cell pairs co-flip 7 times, against about 1.4 expected under independence, and 6 of the 7 co-flips go in the same direction. None of the joint flips has an identical quantized action across cells. These 7 co-flips come from only **5 distinct states** in one block. On q8 U2B, the 5 flipping states each flip under a single cell. We therefore read this as a lead toward *state-level* fragility, where different actions at the same state flip the outcome, and not yet as a finding. [TODO BANKU-F023: per-sub-bank M, taxonomy and co-flip counts at blocks f01–f03 with block intervals, from `analysis/fork/analyze_bank_u.py`]

Common continuation Q0. Under Q0, every cell's BF16 branch has the identical outcome on all 46 shared states, which checks determinism. Q0 turnover is at least the native turnover on the same rows; for example, q4 AWQ-W4 U2B gives 3/21 versus 1/21. With $n = 6$–21 per cell and sub-bank, this is too small to attribute consequences to the controller. [TODO Q0: common-controller turnover per cell at f01–f03; whether action effects compared under Q0 differ across quantizers]

Pending controls. Most consequential differences are text, so the user simulator is a plausible mediator. Simulator choice moves τ-bench outcomes by 9–15 points (Seshadri et al., 2026; Jain and Sandhu, 2026). [TODO SIMCTRL: re-branch a frozen subset of Bank U class-(ii) flips with a second user-simulator model; report the share of flips that persist, against a threshold fixed before the run]. [TODO TOKEN: token-level entropy, KL and margin at the first divergent token of a_B vs. a_Q, flip vs. no flip; a trigger frozen on f01 and evaluated on f02/f03 against a matched-rate random trigger; reported only if it holds on held-out blocks]. Until these controls are in, the claim that consequences depend on the state source, on the full language action and on the continuation controller remains a hypothesis.

Ctrl	Contrast	resc./dam.	turnover	net [95% CI]
B	INV vs VAL	4/3	7/30	$+0.03$ $[-0.13, +0.20]$
B	PP vs VAL	3/2	5/30	$+0.03$ $[-0.10, +0.20]$
B	INV vs PP	2/2	4/30	0.00 $[-0.13, +0.13]$
Q	INV vs VAL	2/2	4/30	0.00 $[-0.13, +0.13]$
Q	PP vs VAL	3/2	5/30	$+0.03$ $[-0.10, +0.17]$
Q	INV vs PP	2/3	5/30	-0.03 $[-0.20, +0.10]$
Controller dependence D ($\neq 0$ in 8/30 states)				$+0.03$ $[-0.17, +0.27]$

Table 4: Defect injection on 30 frozen q4 states (development, one block): per-state rescue/damage of one arm against another, under the BF16 (B) or quantized (Q) continuation controller. $D = [Y(\text{INV}, B) - Y(\text{VAL}, B)] - [Y(\text{INV}, Q) - Y(\text{VAL}, Q)]$. Task-clustered bootstrap intervals; all exact McNemar $p \geq 0.25$. Source: `analysis/recovery/inject_q4/summary_v2.md`.

6 Layer 3: Bounding the "Local Defect $\rightarrow$ Failure" Story

A natural reading of the atlas is that a quantized agent makes a local error, and the task then fails unless the continuation recovers. We test this reading in two ways. First, a controlled intervention injects defects at frozen states. Second, a process audit inspects fork branches. Both results are *bounding*: they limit what the simple story can explain, but they do not identify a mechanism.

Controlled defect injection. We froze 30 ordinary states from q4 GPTQ-W4 trajectories, all from one development block. At each state we forced one of three actions as the next step: an INValid action, a matched VALid action, or a process-perturbation control (PP) that repeats the last successful read. Either B or Q then continued. The defect categories were wrong_id, wrong_tool and premature_write, with 10 states each. Every state passed a replay audit of 22 recorded fields (VAL$\times$Q against the original episode). Three fields were not recorded: the counters right after the restore, the rendered-prompt hash sequence and the tool-schema hash sequence.

Result: two-sided turnover, no error-specific effect. Injected defects change the outcome in 7/30 states under B (4 rescues, 3 damages) and 4/30 under Q (2 and 2). The process-perturbation control changes 5/30 under both controllers (Table 4). The error-specific effect, INV minus PP, is 0.00 $[-0.13, +0.13]$ under B and -0.03 $[-0.20, +0.10]$ under Q. The controller dependence D is nonzero in 8 of 30 states, but its mean is $+0.03$ $[-0.17, +0.27]$. So injected defects cause heterogeneous two-sided turnover of the size a generic

perturbation causes. We detect no error-specific effect beyond that perturbation and no mean controller difference, at $n = 30$ states. An earlier reading of the same data, that injected defects "seldom change outcomes", was wrong and has been retracted.

Manipulation checks limit the categories. Two of 30 states failed a manipulation check: an injected write was rejected, and a valid reference itself returned "not found". Restricting to the 28 passing states leaves the reading unchanged. The wrong_tool category mostly produced argument errors: 8 of 10 injected calls failed with an unexpected-keyword error. We therefore call it a *tool-call/schema error perturbation*, not a wrong-tool test. premature_write is mostly failure→failure: the valid arm under Q already fails in 7/10 of these states, and turnover is 0/10 under both controllers. The event chain error → correction → related requirement satisfied completes in only 6 (B) and 7 (Q) of 29 injected errors, although 17 are corrected under each controller. Injections add tool errors ($+0.43$ under B, $+0.57$ under Q per branch). [TODO RECOVERY-Q8: the same table for q8 GPTQ-W4 (30 states, block q3), from `analysis/recovery/analyze_inject.py` on `analysis/recovery/inject_q8`]

Process audit (LLM-annotated). We sampled 80 development fork states, 20 per outcome sign. Two independent Sonnet 5.5 instances annotated them blind, a third instance adjudicated, and rule-based programmatic labels were added. These annotations are a *process description* conditional on outcome-defined groups. They are not causal evidence, not prevalences and not human labels. Three patterns stand out (Appendix C). First, local defects and final failure do not correspond one-to-one. In damage states, where Q's action led to success and B's to failure, Q's first-action defect is corrected later in 10 of 20 QQ branches. Second, whether a write executes does not separate the failing from the succeeding continuation of the same state (rescue 17 vs. 17; damage 15 vs. 14). Third, the failing continuation more often carries the rubric label `unintended_or_unconsented_change` (rescue: QQ 9 vs. BQ 2, paired difference $+0.35$ $[+0.08, +0.57]$; damage: 1 vs. 9, -0.40 $[-0.62, -0.11]$). In damage states, 7 of these 20 BQ branches carry the label on writes *with* recorded consent. That is the semantic "wrong change" part of the label, and it overlaps with the database-defined outcome by construction. Loops and budget exhaustion also mark failing branches (rescue: QQ 5 vs. BQ 0). We make no claim about consent. The programmatic consent field agrees poorly with the annotators, so we use it only as a mechanical row. Re-labelling 30 states with fresh instances of the same model family reproduces the key pattern. That shows reproducibility, not correctness.

7 Discussion

How action changes become consequences. Replacing a quantization-induced action at the first divergence changes the outcome in 15.5–20.9% of W4 task-block units. It matters in both directions, so the gain from replacing the action at the fork is small on average. Which way a change goes depends on the state. The concentration of flips in fragile tasks, the co-flips of different quantizers with different actions at the same state, and the dominance of text differences all point to properties of the state and of the full language action. They do not point to a stable "quantization error" that a later controller repairs or not. The injection study and the process audit bound the alternative story: an injected local defect is about as consequential as a generic process perturbation, and failing continuations differ by *wrong* changes and loops rather than by more writes.

Implications for evaluating quantized agents. (1) Report r, d and M next to any net quantity. A small G_{all} coexisted with 15.5–20.9% turnover. (2) Keep the forked-state gain apart from the end-to-end success comparison; they answer different questions, and the latter needs its own paired, task-resampled interval (Appendix B). (3) Treat language-only differences as actions. At ordinary states they carry most consequential differences, and tool-call metrics do not see them.

What we do not claim. We do not claim success equivalence between BF16 and W4 agents. Equivalence tests at ± 3 pp fail. We do not claim a W×KV interaction, whose pre-registered replication was not supported. We do not present static precision restoration as a method, and we do not claim that "errors are absorbed" (Appendix E). G_{all} is a retrospective oracle quantity, since finding the fork state requires running both models. It is not the value of a deployable escalation policy.

Limitations

One model family, two sizes. All agents are Qwen3 models (4B and 8B). Effects may differ at other scales and in other families. [TODO Q14: replace or extend with Qwen3-14B paired forks if they finish; a second family is best effort and is not claimed]

One user simulator. All τ^2-bench episodes use a single greedy simulator model, and continuation effects include its replies. Most consequential differences at ordinary states are text, so they may partly reflect this simulator rather than the task ([TODO SIMCTRL: second-simulator persistence result]).

LLM annotation. Process labels come from LLM annotators (Sonnet 5.5, two independent instances plus an adjudicator) and rule-based checks. No human annotation has been done. Agreement among instances of the same model family shows reproducibility, not correctness.

Simulated quantization. Quantized checkpoints are dequantized and served with BF16 compute on a deterministic path. Deployed low-bit kernels change the numerics and add serving variation that we deliberately remove. We make no claims about speed, memory or deployed kernels.

Small and partial confirmation. The preregistered holdout covers 34 telecom tasks and three cells. Few events there would mean limited evidence, not absence. Even a positive confirmation would not generalise beyond telecom or beyond these cells. The fragility results in §4 remain a retrospective association on development data until then.

Scope of the design. The atlas intervenes only at the first non-text divergence per task and block. Bank U samples one state per task and so far one block. The defect-injection study uses one model and 30 states. Prompt nonces are a researcher-chosen perturbation that defines replicate blocks; they are not a natural deployment distribution.

Ethics Statement

This work involves no crowdworkers and no personal data. All users are simulated by an LLM on public benchmark tasks, and all agents are openly released Qwen3 checkpoints. The process annotations were produced by LLMs on synthetic benchmark conversations. The main risk we see is overreading. A small forked-state gain on one simulator and a fixed task pool does not certify a quantized agent for deployment, and our end-to-end comparisons do not establish equivalence. Deployment decisions in consequential settings should not rest on these measurements alone.

References

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. 2026. On randomness in agentic evals. *arXiv preprint arXiv:2602.07150*.

Yihang Chen, Pin Qian, Su Wang, Chong Peng, Huan Xu, Xiyang Wu, and Yiqi Sun. 2026. Noise floor audit for agent benchmarks. *arXiv preprint arXiv:2608.22331*.

Peijie Dong, Zhenheng Tang, Xiang Liu, Lujun Li, Xiaowen Chu, and Bo Li. 2025. Can compressed LLMs truly act? an empirical evaluation of agentic capabilities in LLM compression. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*.

Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. 2024. Accuracy is not all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*.

Ashritha Gonuguntla. 2026. The replay gap: Static evaluation of model switching in LLM agents scores the wrong world. *arXiv preprint arXiv:2608.08239*.

Ashish Jain and Armaan Sandhu. 2026. UserProxyBench: Evaluating LLM user simulators for agent benchmarks and training. *arXiv preprint arXiv:2609.38043*.

Jiwon Jang, Kisu Yang, Heuiseok Lim, and Hyunwoo Park. 2026. Flat score, amplified failures: How the error budget masks damage in quantized LLM agents. *arXiv preprint arXiv:2607.27275*.

Mukai Li, Qingcheng Zeng, Tianqing Fang, Zhenwen Liang, Linfeng Song, Qi Liu, Haitao Mi, and Dong Yu. 2026a. Verified critical step optimization for LLM agents. *arXiv preprint arXiv:2602.03412*.

Yuanhao Li, Hongbo Wang, Xuhong Chen, Yiming Cao, and Xunzhu Tang. 2026b. Counterfactual rollout replay: Forkable environments as free process rewards for software engineering agents. *arXiv preprint arXiv:2609.33875*.

Yuanzhe Li, Jianing Deng, Jingtong Hu, Tianlong Chen, Song Wang, and Huanrui Yang. 2026c. Dynamic mixed-precision routing for efficient multi-step LLM interaction. *Preprint*, arXiv:2602.02711.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*.

Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. 2026. Quantized reasoning models think they need to think longer, but they do not. *arXiv preprint arXiv:2606.00206*.

Haiquan Lu, Zigeng Chen, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2026. Mix-Quant: quantized prefilling, precise decoding for agentic LLMs. *arXiv preprint arXiv:2605.20315*.

Aditi Patodiya. 2026. Same request, different answer: Quantization amplifies Cache-Induced divergence in LLM serving. *arXiv preprint arXiv:2609.04748*.

Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. 2026. Lost in simulation: LLM-simulated users are unreliable proxies for human users in agentic evaluations. *arXiv preprint arXiv:2601.17087*.

Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2025. AgenTracer: Who is inducing failure in the LLM agentic systems? *arXiv preprint arXiv:2509.03312*.

Manyi Zhang, Ji-Fu Li, Zhongao Sun, Xiaohao Liu, Zhenhua Dong, Xianzhi Yu, Haoli Bai, and Xiaobo Xia. 2026. QuantClaw: precision where it matters for OpenClaw. *arXiv preprint arXiv:2604.22577*.

A Setup Details

Splits. The development split has 58 retail, 27 airline and 80 telecom tasks. The holdout has 34 telecom tasks, and its pre-registration hash is recorded in the split file. The telecom split was fixed from τ^2-bench's 114-task base split by a stratified, salted hash before any run. The external benchmarks use BFCL multi-turn (base, miss_func, miss_param and long_context, 197 items each) and ALFWorld (134 out-of-distribution games). Items were selected by hash in manifests committed before any model run.

Success endpoints. For retail and airline we use the final-database match with normal termination. This is not τ^2-bench's official reward, which additionally scores natural-language or communicate checks. Telecom uses the official deterministic task-basis success. Context overflow and unparseable output count as failures. Infrastructure outages are missing, never failures, and a block with a missing unit leaves the primary analysis.

Fork mechanics. The forced action is returned by a fork agent for the one step that follows h_t. A tool call is executed by the real environment, and text is answered by the real simulator, which never reuses observations from the original trajectory. tau2 resets step and error counters when it starts from a history. The fork restores them to the values used at t, so the remaining budget equals the original. Every max_steps and too_many_errors episode on the checked blocks is reproduced by the recount.

Composition of divergent states. Table 5 gives the four-way outcome composition behind Table 1.

Cell	units	div. states	rescue	damage	both succ.	both fail
q4 GPTQ-W4	1700	1102	116	148	267	571
q4 AWQ-W4	1700	1439	167	172	340	760
q8 GPTQ-W4	1700	1473	177	128	276	892
q8 AWQ-W4	1700	1562	194	161	318	889
q4 KV8	1870	593	58	64	154	317
q8 KV8	1870	855	81	70	161	543

Table 5: Divergent-state composition of $(Y_{\mathrm{BQ}}, Y_{\mathrm{QQ}})$ on the τ^2-bench panel (85 tasks; 20 blocks for W4, 22 for KV8). Units without a divergence count as zero in r, d, M and G_{all}. Source: `analysis/fork/out/dev_all_20261001T070326Z.md`.

Cell	A	C	I	strict rev. (%)
q4 GPTQ-W4	$-3.2\,[-6.4, +0.2]$	$+0.5\,[-1.2, +2.2]$	$+0.3\,[-3.1, +3.4]$	2.6
q4 AWQ-W4	$-1.7\,[-3.6, +0.4]$	$-0.4\,[-2.4, +1.5]$	$+1.3\,[-1.6, +4.0]$	3.1
q8 GPTQ-W4	$+5.0\,[+3.2, +6.8]$	$+2.1\,[+0.2, +4.0]$	$-1.7\,[-4.1, +0.7]$	2.7
q8 AWQ-W4	$+1.9\,[-0.1, +3.8]$	$-1.5\,[-4.2, +0.8]$	$+0.3\,[-3.5, +4.1]$	4.2
q4 KV8	$+1.3\,[-2.5, +4.9]$	$-0.2\,[-3.5, +3.7]$	$-2.4\,[-6.2, +1.2]$	1.2
q8 KV8	$+2.8\,[+0.3, +5.5]$	$+1.2\,[-0.5, +3.2]$	$-1.5\,[-4.3, +0.9]$	1.2

Table 6: Four-branch decomposition on divergent states (pp; block-bootstrap 95% intervals). $A = Y_{\mathrm{BB}} - Y_{\mathrm{QB}}$, $C = Y_{\mathrm{BB}} - Y_{\mathrm{BQ}}$, $I = Y_{\mathrm{BQ}} + Y_{\mathrm{QB}} - Y_{\mathrm{BB}} - Y_{\mathrm{QQ}}$. Strict reversal: $A \cdot G < 0$. Source: `analysis/fork/out/dev_all_20261001T070326Z.md`.

B End-to-End Success Comparison

The forked-state gain G_{all} is not an end-to-end comparison. Table 7 gives the end-to-end comparison, computed separately on the same checkpoints,

the same tasks and the same 20 nonce blocks as the forks.

Cell	Q / BF16 (%)	Q−BF16 [95% CI]	NI 3 pp	TOST ±3 pp
q4 GPTQ-W4	40.0 / 35.8	+4.2 [−2.4, +10.5]	pass	fail
q4 AWQ-W4	36.8 / 35.8	+0.9 [−5.8, +7.5]	fail	fail
q8 GPTQ-W4	30.1 / 30.4	−0.2 [−7.7, +6.8]	fail	fail
q8 AWQ-W4	31.5 / 30.4	+1.2 [−5.3, +7.6]	fail	fail
pool, all four		+1.5 [−2.7, +5.6]	pass	fail

Table 7: End-to-end closed-loop success, quantized minus BF16 (pp). Units are paired on nonce, and the crossed task×block bootstrap is drawn jointly across cells. Non-inferiority (NI) at a 3 pp margin passes for q4 GPTQ-W4 and the all-four pool. Equivalence (TOST at ±3 pp) passes nowhere. These comparisons therefore support neither a difference nor equivalence. Source: `analysis/insights/iter_4/out/success_ni_ALIGNED_20260930T213009Z.md`.

C Process Audit (Table 4 v2)

The core "local defect" combines groundedness, argument validity and permission/consent judgements. The tool-validity field (inter-annotator $\kappa = 0.29$) is excluded from it. Mechanical fields agree with the programmatic verifier (write executed vs. the rubric's irreversible-change field, $\kappa = 0.99$). The consent fields agree only moderately ($\kappa = 0.44$ on writes both see).

D Holdout Pre-Registration

The protocol `docs/insights/PREREG_FORK_CONFIRM.md` was frozen before any holdout unit was run. Its sha256 is `902892dd63e99e16ab9beefee9d39319d835eacdada3d4d92c4a1022d13fd622`. It fixes the following design:

- tasks, cells, weights and nonce blocks;
- halves $H_A = \{h01, h04, h05, h09, h11, h12\}$ and H_B, the rest of $h01$–$h12$;
- an outcome-blind repair rule, with reserve blocks used only for infrastructure repair;
- the decisional crossed interval (two-way ANOVA moments, normal 90%);
- the readings: positive if $L > 0$, above 3 pp if $L > 3$, small if the interval lies inside ±3 pp, otherwise inconclusive;
- the secondary families S1 (both $r > 0$ and $d > 0$) and S2 (persona-stratified permutation, 10,000 draws, no outcome screen);
- six stratum families, with Holm correction within each.

The analysis script refuses holdout data unless the hash matches. [TODO HOLDOUT: full confirmatory table (per cell and pooled: units, exclusions, r, d, M, G_{all} with crossed and block-t intervals; S2 ρ_c; strata)]

E Withdrawn and Negative Results

W×KV interaction (not supported). An earlier q4 analysis reported a W4×KV4 interaction of -14.9 [-23.8, -6.0] pp ($k = 3$). Its KV16 reference lacked the Q/K rotation used for KV4, so rotation and quantization were confounded. With a matched rotated reference, the q4 interaction is -9.0 [-30.6, $+12.5$]. The pre-registered q8 replication gives $+2.1$ [-5.8, $+10.1$] over 6 blocks, crossed [-8.2, $+12.4$]. The replication is not supported, and we make no W×KV claim.

Static precision restoration (appendix only). We tested whether restoring selected modules to higher precision recovers a damaged RTN-W3 model. We report this line as a mechanism stress test, not a method. In RTN-W3 the gains largely tracked the number of restored bits. A random MLP arm also recovered much of the gap, and no arm beat whole-model GPTQ-W4 at matched storage. [TODO RESTORE: the q8 RTN-W3 attention vs. byte-matched random-attention result (one seed) is cited only after its restoration output is committed; it is not used for any claim]

Success equivalence. Not established (Table 7).

"Errors are absorbed". An interim reading of the injection study said that injected defects seldom change outcomes. The per-state analysis shows two-sided turnover (§6), so this reading is retracted.

Label (source)	Comparison	rescue	damage	both succeed	both fail
`unintended_or_unconsented_change` (LLM)	QQ vs BQ	9 vs 2, $+0.35$ [$+0.08$, $+0.57$]	1 vs 9, -0.40 [-0.62, -0.11]	1 vs 1	5 vs 7, -0.10 [-0.35, $+0.17$]
write executed (programmatic)	QQ vs BQ	17 vs 17, 0.00 [-0.25, $+0.25$]	15 vs 14, $+0.05$ [-0.26, $+0.35$]	10 vs 9	11 vs 13
executed write w/o recorded consent (prog.)	QQ vs BQ	9 vs 5, $+0.20$ [-0.06, $+0.43$]	3 vs 3, 0.00 [-0.21, $+0.21$]	3 vs 2	4 vs 3
loop or budget exhaustion (LLM)	QQ vs BQ	5 vs 0, $+0.25$ [$+0.04$, $+0.47$]	0 vs 2, -0.10 [-0.30, $+0.08$]	1 vs 2	7 vs 4
action judged indefensible (LLM)	Q vs B action	7 vs 8, -0.05 [-0.37, $+0.28$]	12 vs 5, $+0.35$ [-0.05, $+0.65$]	10 vs 6	12 vs 9
local defect, core (LLM)	Q vs B action	4 vs 6, -0.10 [-0.33, $+0.14$]	12 vs 7, $+0.25$ [-0.09, $+0.53$]	6 vs 3	11 vs 7
QQ first-action defect corrected later (LLM)	single branch	4/20 [0.08, 0.42]	10/20 [0.30, 0.70]	4/20	7/20

Table 8: Process audit of 80 development fork states, 20 per outcome sign (rescue: $Y_{\mathrm{BQ}} = 1, Y_{\mathrm{QQ}} = 0$; damage: $Y_{\mathrm{BQ}} = 0, Y_{\mathrm{QQ}} = 1$). **LLM-annotated** by Sonnet 5.5 (two independent blind instances and an adjudicating instance), plus rule-based programmatic labels; no human labels. Counts are first vs. second of $n = 20$ states. Intervals are within-state paired differences with Newcombe 95% CIs; single-branch rows use Wilson 95% intervals. This is a process description conditional on outcome-defined groups: not causal, not prevalence. The label name is the rubric category; we make no claim about consent. Source: `analysis/annotation/wp5/table4_v2.md`.